

LLM Deployment

CS 541 Lecture 8

Zining Zhu

Where we left off

Last week was about **shaping** a model's behavior after pretraining.

- Lecture 7 covered SFT, RLHF (PPO), DPO, and RLVR (GRPO) — all about **what** the model says
- Today is about **using** a trained model: grounding it with retrieval, letting it act as an agent, then the systems engineering of serving it
- Same model, radically different cost depending on how it is served

Today's roadmap

- **RAG**: giving a frozen model fresh, grounded knowledge at request time
- **Agents**: models that act in a loop, not just answer once
- **Serving constraints**: memory, compute, and the latency/throughput tradeoff behind every deployment decision
- **The KV cache**: why generation is fast, and why it eats memory
- **Parallelism**: splitting a model across devices when it doesn't fit on one
- **Memory optimization**: activation checkpointing, mixed precision, quantization

By the end, RLHF from last week will look like a special case of a much bigger idea: models as **components** in larger systems.



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Retrieval-Augmented Generation (RAG)

Motivation · Architecture

Why retrieval?

A model's knowledge is frozen at training time.

- **Knowledge freshness**: the model can't know about anything after its training cutoff
- **Hallucination**: models generate fluent, confident, and sometimes wrong answers, with no built-in way to say "I don't actually know this"
- Retraining or fine-tuning to add new knowledge is slow and expensive — and the wrong tool for facts that change daily
- **RAG**: fetch relevant text at request time, and hand it to the model as context, instead of baking facts into weights

RAG architecture

- **Retriever**: given a query, find the most relevant documents from a corpus
- **Generator**: the LLM, now conditioned on both the query **and** the retrieved documents
- Retrieval typically uses **embeddings**: dense vectors where similar meaning maps to nearby vectors
- A **vector database** stores document embeddings and supports fast nearest-neighbor search over millions or billions of them

Embeddings, recap and application

- Recall lecture 6: language models already learn dense representations of text internally
- A text **embedding model** is trained so that semantically similar text produces nearby vectors, and dissimilar text produces distant vectors
- This is the same “learning from data” thread as the rest of the course, now used as a **retrieval** tool rather than a generation tool
- Cosine similarity (or dot product) between two embeddings approximates how related their underlying texts are

Vector databases and approximate search

- Exact nearest-neighbor search over millions of vectors is too slow for interactive use
- **Approximate nearest neighbor (ANN)** search trades a small amount of accuracy for large speedups
- **HNSW** (Hierarchical Navigable Small World): a layered graph of vectors; search hops greedily toward the query, coarse layers first
- **IVF** (Inverted File index): cluster the vectors, then search only the few clusters nearest the query
- A vector database wraps one of these structures with storage, filtering, and update support

Example: one query through the RAG pipeline

1. User asks: “What did the Fed decide about interest rates this week?”
2. **Embed** the query into a vector using the same embedding model used for the corpus
3. **Search** the vector database for the top-k most similar document chunks (e.g. recent news articles)
4. **Construct** an augmented prompt: the retrieved chunks plus the original question, formatted for the generator
5. **Generate**: the LLM produces an answer conditioned on both the question and the retrieved evidence

Practical consideration: chunking

- Documents must be split into **chunks** before embedding — a whole book doesn't fit one embedding well, and burying one relevant sentence in thousands of irrelevant ones dilutes the match
- Chunk size is a tradeoff: too small loses surrounding context, too large dilutes relevance and wastes prompt budget on irrelevant text
- Two common strategies: **fixed-size** chunking (e.g. every 500 tokens) is simple but can cut ideas in half; **semantic chunking** splits at natural boundaries (paragraphs, sections) but needs extra processing

Example: chunking the same document two ways

- A three-paragraph policy document, one idea per paragraph
- **Fixed-size (300 tokens)**: cuts straight through paragraph 2, splitting one idea across two chunks — a query about that idea might retrieve only half of it
- **Semantic (by paragraph)**: each chunk is one complete idea — better retrieval precision, at the cost of variable chunk sizes that complicate batching

Chunk overlap

- Overlapping chunks (e.g. each chunk shares its last 50 tokens with the next chunk's first 50) help avoid cutting a relevant idea exactly at a chunk boundary
- Overlap increases storage and embedding cost roughly in proportion to the overlap fraction
- A small overlap (10-20%) is a common default: enough to catch boundary-spanning ideas, not so much that most of the corpus is duplicated

Practical consideration: reranking

- Initial retrieval (fast, approximate, embedding-based) often returns a shortlist that isn't perfectly ranked by **actual** relevance
- A **reranker**: a slower, more accurate model (often a cross-encoder that looks at the query and each candidate **together**, not as separate embeddings) that re-scores the shortlist
- Two-stage retrieval — fast recall of many candidates, then accurate rerank of a short list — balances speed and quality

Example: two-stage retrieval

- Stage 1 (recall): ANN search returns the top 100 candidate chunks in milliseconds, embedding-based
- Stage 2 (rerank): a cross-encoder scores all 100 candidates against the query directly, which is too slow to run over the whole corpus but fine over 100 items
- Only the reranked top 5 go into the generator’s prompt
- This pattern — cheap-and-broad, then expensive-and-narrow — recurs throughout search and retrieval systems, echoing lecture 4 and 5’s search algorithms

Evaluating RAG systems

- **Retrieval metrics**: recall@k (is the relevant chunk anywhere in the top k results?), precision@k
- **Generation metrics**: faithfulness/groundedness (does the answer actually follow from the retrieved text, or did the model add unsupported claims?)
- A RAG system can fail at either stage independently — good generation over bad retrieval still produces a wrong (if fluent) answer
- Diagnosing **which** stage failed is the first step in debugging a RAG system in production

RAG failure modes

- **Retrieval failure**: the right chunk was never surfaced — happens with vocabulary mismatch between query and document, or a corpus that lacks the answer entirely
- **Generation failure**: the right chunk was retrieved, but the model ignored it or contradicted it — the deeper “hallucination despite evidence” problem
- **Stale index failure**: the corpus or its embeddings weren’t updated, so retrieval returns confidently wrong (outdated) information
- Each failure mode calls for a different fix — more retrieval candidates, a better reranker, a better generator prompt, or a fresher index

RAG vs. fine-tuning vs. long-context

Approach	Best for	Weakness
RAG	**Frequently-changing facts, large corpora**	Retrieval can miss or misrank
Fine-tuning	**Stable style/behavior changes**	Slow, expensive, doesn't fix freshness
Long-context	**Small, static corpora**	Expensive per request (see the KV cache section later today); attention can still miss details

Agentic RAG, a pointer

- So far, retrieval happens once, automatically, before generation
- An **agentic** version lets the model **decide** whether to retrieve, what to search for, and whether to search again after seeing the first results
- This blurs into the next section's topic — a model that takes actions (like "search again") in a loop, rather than following one fixed pipeline
- We'll pick this thread back up as soon as we introduce agents properly

Check your understanding

- Why can a RAG system still produce a wrong answer even when the retriever surfaces the correct document?
- What's the tradeoff between fixed-size and semantic chunking, and why does chunk overlap partially address it?
- When would long-context prompting be a **better** choice than RAG, given that every extra prompt token costs compute and memory? (Revisit this after the KV cache section.)

Section recap

- RAG separates **what a model knows** (fixed at training time) from **what a model has access to** (updated at request time)
- The pipeline is retriever → generator, built on embeddings and vector search, refined with chunking strategy and reranking
- RAG fixes what facts are in front of the model — it does not fix the model's reasoning
- Next: models that don't just answer once, but act, observe, and act again



STEVENS
INSTITUTE OF TECHNOLOGY
1870

LLM agents

Tool use · ReAct · Multi-agent

From answering to acting

- So far in this course: the model reads a prompt (possibly augmented by RAG) and produces text, once, and stops
- An **agent**: a model that can take **actions** in a loop, observe the results, and decide what to do next
- Actions might be: calling a search API, running code, querying a database, calling another model
- The retrieval decision from the end of the last section — “should I search again?” — is exactly this kind of action

Tool use / function calling

- The model is given a set of tools, each with a name, description, and expected arguments (much like a function signature)
- The model decides **which** tool to call and **what** arguments to pass, based on the request: it outputs a structured call rather than free text
- The tool executes outside the model (an actual API call, a code sandbox, a database query); the result is fed back to the model as new context
- This turns a language model into something that can interact with the outside world, not just describe it

Telling the model which tools exist

- Each tool is described by a **JSON schema**: a name, a plain-language description, and typed parameters
- The serving layer renders these schemas into the prompt text, through the model's **chat template**
- The model only ever sees tokens, so the schema is just more context to condition on
- Descriptions matter: the model chooses a tool by reading them

How the model emits a call

- No new mechanism: a call is ordinary next-token prediction, learned in post-training (SFT and RL on tool-use transcripts, as in lecture 7)
- Many model families reserve **special tokens** to mark a call: a start-of-call token, a JSON payload, an end-of-call token
- Special tokens are vocabulary entries that never occur in normal text, so the serving code can detect them reliably
- Exact tokens differ by model; some models emit plain-text JSON or a code block in an agreed format instead

Example: a weather tool call

1. User asks: “Should I bring an umbrella to Hoboken tomorrow?”
2. The model decides this requires current data it doesn’t have, and calls `get_forecast(location="Hoboken, NJ", date="tomorrow")`
3. The tool executes against a real weather API and returns: `{"condition": "rain", "probability": 0.8}`
4. This result is appended to the model’s context
5. The model generates its final answer: “Yes, 80% chance of rain in Hoboken tomorrow.”

Example: the same call, as text in the context

Role	Text the model reads or writes
user	Should I bring an umbrella to Hoboken tomorrow?
assistant	`<tool_call>{"name": "get_forecast", "arguments": {"location": "Hoboken, NJ", "date": "tomorrow"}}</tool_call>`
tool	`{"condition": "rain", "probability": 0.8}`
assistant	Yes, 80% chance of rain in Hoboken tomorrow.

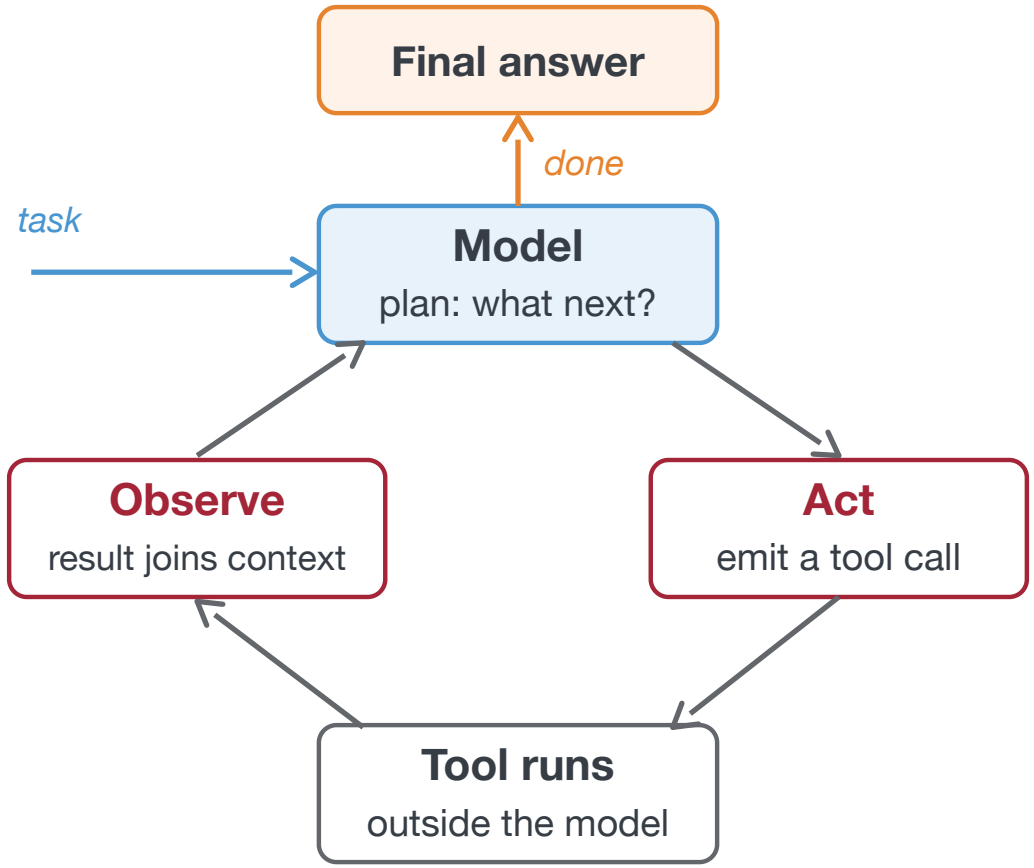
Illustrative: the tag names and role markers are special tokens in a real model, and they differ by model family.

What runs the call: the harness

- Generation stops when the model emits its end-of-call token
- The harness parses the JSON, checks it against the schema, and runs the real function
- The result goes back as a new message with a tool role, and generation resumes
- The model never executes anything: it writes text, and ordinary code acts on it
- Optional **constrained decoding** masks tokens that would break the schema, so the JSON is always valid

The agent loop

- **Plan**: the model decides what to do next given the current state
- **Act**: it calls a tool, or produces a final answer if the task is done
- **Observe**: the result becomes new context
- Repeat until done: every agent framework is a variation on this loop



ReAct: reasoning + acting

- **ReAct**: interleave explicit reasoning (“thought“) steps with actions in the same loop
- The model writes out **why** it’s taking an action before taking it, rather than jumping straight to a tool call
- Interleaving reasoning and acting improves reliability over acting without an explicit reasoning step — the model can catch its own mistakes mid-trajectory
- Still fundamentally the plan-act-observe loop, made explicit in the model’s own output

Example: a full ReAct trace

Task: “How many years older is the current Stevens president than the university itself?”

1. **Thought**: I need the university’s founding year and the president’s birth year.
2. **Action**: search(“Stevens Institute of Technology founding year“)
3. **Observation**: “Founded in 1870.”
4. **Thought**: Now I need the current president’s age or birth year.
5. **Action**: search(“Stevens Institute of Technology current president birth year“)
6. **Observation**: “Born in 1969.”
7. **Thought**: University age in 2026 is 156 years; president’s age is about 57. The question asked something different than I can answer this way — I should recompute what’s actually being asked.
8. **Final Answer**: reformulate and answer based on the corrected reasoning.

Why interleaving reasoning helps

- Step 7 above shows the point: writing out reasoning let the model **notice** its own confusion mid-trajectory, before committing to a wrong final answer
- Without an explicit thought step, the same error might have propagated silently into a confidently wrong answer
- This is the same instinct behind chain-of-thought prompting (lecture 6) — visible intermediate reasoning is easier to check, both for the model and for a human reviewing the trace
- The cost: more tokens generated per task, since reasoning steps are not free

Failure modes of agents

- **Infinite loops**: the model repeats a failing action without recognizing it isn't working
- **Hallucinated tool calls**: calling a tool that doesn't exist, or with arguments the tool can't accept
- **Compounding errors**: an early wrong observation (e.g. a bad search result) poisons every reasoning step that follows it
- **Runaway cost**: each loop iteration costs tokens and possibly real-world side effects (an API call, a purchase, a sent email) — unlike a single generation, mistakes can have consequences outside the chat window

Guardrails for agents that take real actions

- Restrict which tools an agent can call for a given task (least privilege, same principle as security engineering generally)
- Require confirmation before **irreversible** actions (sending an email, making a payment, deleting a file)
- Cap the number of loop iterations, so a stuck agent fails loudly instead of running (and spending) indefinitely
- Log every action and observation — the trace itself is the debugging tool when something goes wrong

Multi-agent orchestration

- Some tasks are split across **multiple** agents, each with a role (e.g. a planner, a coder, a reviewer)
- Agents communicate by passing messages or artifacts to each other, rather than one agent doing everything itself
- Motivation: specialization (a “reviewer” agent prompted only to critique often catches more than a single agent asked to write-and-check itself) and parallelism (independent subtasks can run at once)
- An active and fast-moving research area — not a solved problem, and conventions vary a lot between systems

Example: a planner-worker trace

Task: “Summarize this codebase’s authentication logic and its test coverage.”

1. **Planner agent**: splits the task into two subtasks — “find and summarize the auth code” and “find and summarize the auth tests” — and dispatches one worker per subtask
2. **Worker A**: reads the authentication module, returns a summary of its logic
3. **Worker B**: reads the test files, returns a summary of what’s covered and what’s missing
4. **Planner agent**: receives both summaries, merges them into one coherent report, noting any gap between what the code does and what the tests check

Evaluating agents

- **Task success rate**: did the agent actually accomplish the goal, not just produce plausible-looking output?
- **Cost**: number of tool calls, tokens generated, and wall-clock time to complete the task
- **Safety**: did the agent stay within its intended scope, or did it take unintended or irreversible actions?
- Unlike evaluating a single generation, agent evaluation has to account for an entire **trajectory**, not one output

Check your understanding

- Why does writing an explicit “thought” step before each action tend to reduce compounding errors, compared to acting directly?
- What’s the difference in risk profile between a single-agent tool call and a multi-agent system where agents can trigger each other’s actions?
- Name one guardrail you’d insist on before letting an agent you built send real emails on a user’s behalf.



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Serving LLMs at scale: the constraints

Memory · Compute · Latency

Why serving is its own problem

Training happens once; serving happens every request, forever.

- A trained model is only useful if it can answer requests **fast enough** and **cheap enough**
- Serving introduces constraints training doesn't have: strict latency budgets, many concurrent users, unpredictable request sizes
- Two resources to manage: **memory** and **compute** — and they bind in different places
- A serving system is really two systems: one that handles the **prompt** (prefill), one that handles **generation** (decode)

Memory pressure: weights

- Model weights must fit in GPU memory just to load the model, before anything else happens
- A 70B-parameter model in 16-bit precision needs roughly 2 bytes/parameter → **~140GB**, before any activations or cache
- Bigger models get split across multiple GPUs simply to fit — this is the first reason parallelism exists, before speed ever enters the picture

Example: weight memory across precisions

Precision	Bytes/param	7B model	70B model	175B model
fp32	4	~28GB	~280GB	~700GB
fp16 / bf16	2	~14GB	~140GB	~350GB
int8	1	~7GB	~70GB	~175GB

A single top-end GPU today has 80-192GB of memory — read the table against that, and multi-GPU serving stops looking optional.

Compute cost in FLOPs

- Each generated token costs roughly $2 \times (\text{number of parameters})$ floating-point operations for a forward pass
- The factor of 2 comes from one multiply and one add per parameter-weight interaction in a matrix multiply
- A 70B model therefore costs ≈ 140 GFLOPs per generated token, **before** accounting for attention over a long context
- This is why parameter count is the single best rough predictor of inference cost

Prefill vs. decode: two different regimes

- **Prefill**: processing the entire prompt at once, computing attention over it in parallel — **compute-bound**, uses the GPU's raw FLOP throughput well
- **Decode**: generating one token at a time, each step re-reading the whole KV cache — **memory-bandwidth-bound**, the GPU sits mostly idle waiting on memory
- Same model, same hardware, two very different bottlenecks depending on which phase you're in
- This split motivates different optimizations for prompt processing vs. generation — batching helps decode much more than prefill

The roofline model, briefly

A simple way to reason about which resource limits you.

- **Arithmetic intensity**: FLOPs performed per byte moved from memory
- If arithmetic intensity is low, you are **memory-bound**: faster compute doesn't help, faster memory does
- If arithmetic intensity is high, you are **compute-bound**: more FLOPs/second directly helps
- Decode has very low arithmetic intensity (few FLOPs per byte read from memory) — this is the formal reason it's memory-bandwidth-bound

Latency vs. throughput

Serving systems face a fundamental tradeoff.

- **Latency**: time for one request to get its response — matters for interactive chat
- **Throughput**: total tokens served per second across all requests — matters for cost per token
- **Batching**: grouping requests together raises throughput but can raise per-request latency
- **Continuous batching**: dynamically add and remove requests from a running batch as they arrive and finish, instead of waiting for a fixed batch to complete together

Continuous batching, in more detail

- Naive batching: wait until B requests have arrived, run them together, wait for **all** to finish before starting the next batch
- Problem: one long request blocks the batch slot for many short ones that finished early
- Continuous batching: at every decode step, slot in newly-arrived requests and evict finished ones, token by token
- This keeps GPU utilization high without forcing every request to wait for the slowest one in its cohort

Metrics that actually matter for serving

Metric	What it measures	Who cares
TTFT (time to first token)	**prefill latency**	interactive chat users
TPOT (time per output token)	**decode speed once streaming starts**	perceived “typing speed”
Throughput (tokens/sec, aggregate)	**total system capacity**	cost per served token
P99 latency	**tail latency, not just average**	SLA compliance

Example: two SLOs, two designs

Use case	Priority	Design choice
Interactive chat assistant	**Low TTFT, steady TPOT**	Small batches, prioritize new requests, maybe smaller model
Offline batch summarization	**Maximize throughput**	Large batches, no latency deadline, biggest batch the memory allows

There is no universally “best” serving configuration — only the right point on the latency-throughput curve for **your** workload.

Check your understanding

- Why does a 70B model in fp16 need more than one 80GB GPU before it has served a single request?
- Which phase of generation, prefill or decode, is compute-bound, and which is memory-bandwidth-bound? Why does batching help one much more than the other?
- Why can a chat service and an offline summarization job justify very different batch sizes on the same model?

Section recap

- Serving is bound by two resources, memory and compute, that bind at different points in the request lifecycle
- Prefill and decode are different regimes with different bottlenecks
- Every serving decision is a point on the latency-throughput curve
- Next: the KV cache, the component that makes decode fast and also makes memory the bottleneck



STEVENS
INSTITUTE OF TECHNOLOGY
1870

The KV cache

Why cache · What to cache · What it costs

Attention, recapped: queries, keys, values

Each token, at each layer, produces three vectors from its hidden state h_i.

- Query $q_i = W_Q h_i$: what token i is looking for
- Key $k_i = W_K h_i$: what token i offers to be matched against
- Value $v_i = W_V h_i$: the content token i passes along when it is matched
- Output at position t: $o_t = \sum_{i \le t} \mathrm{softmax}_i\left(q_t \cdot k_i / \sqrt{d}\right) v_i$, summed only over $i \le t$ because of the causal mask

Generation without a cache

Autoregressive decoding emits one token per step and appends it to the input.

- The naive loop feeds all n current tokens back through the model at every step
- That recomputes q, k, v for all n tokens, plus the full $n \times n$ table of attention scores
- Almost all of it is a repeat: tokens $1, \ldots, n-1$ are exactly what they were one step ago
- Attention cost per step: $O(n^2)$

Key observation: the past never changes

- Causal masking means token i attends only to tokens $\leq i$
- So token i's hidden state at every layer depends only on tokens $1, \ldots, i$, and appending new tokens cannot change it
- Therefore k_i and v_i, at every layer, stay valid for the rest of generation
- Compute them once, store them, reuse them: that store is the **KV cache**

Why K and V are cached, and Q is not

Vector	Who uses it	Needed at later steps?
Query q_i	Only the output at position i	No: that output already exists
Key k_i	Scoring by every **later** query	Yes, at every future step
Value v_i	Weighted sum for every **later** query	Yes, at every future step

Step t needs one new query q_t, matched against all keys $k_{1..t}$ and mixed from all values $v_{1..t}$. Old queries have no remaining consumer, so storing them would only waste memory.

One decode step, with the cache

1. Take the newest token's hidden state h_t at this layer
2. Compute only q_t, k_t, v_t: three small matrix-vector products
3. Append k_t and v_t to this layer's cache
4. Score q_t against all t cached keys, apply softmax, and take the weighted sum of the t cached values
5. Repeat for every layer, then sample the next token

Why the cache cuts the cost

Let n be the current sequence length.

	Without cache	With cache
Tokens projected to q, k, v, per step	n	1
Attention scores, per step	$n \times n$, so $O(n^2)$	$1 \times n$, so $O(n)$
Attention, whole n-token generation	$O(n^3)$	$O(n^2)$

Each step is still $O(n)$, because the new query must look at every past key. The cache removes the repeated work, not the dependence on context length.

Example: counting the savings

Producing token 1000 of a response, one head, one layer.

- Without the cache: $1000 \times 1000 = 10^6$ query-key scores, and 1000 tokens' k, v recomputed
- With the cache: $1 \times 1000 = 10^3$ scores, and one new k, v pair: **1000× fewer** scores
- Whole 1000-token generation: $\sum_t t^2 \approx 3.3 \times 10^8$ scores versus $\sum_t t \approx 5 \times 10^5$
- That is roughly **670× less** attention work, and the gap grows with length

The price: memory instead of compute

- The cache trades repeated compute for stored state, a classic space-for-time exchange
- Every layer keeps its own K and V for every past token, all resident in GPU memory
- It grows by one entry per generated token, per request, for as long as the request lives
- Decode now spends its time **reading** the cache, which is why it is memory-bandwidth-bound

The KV cache size formula

For one request, one token position, the cache holds one key vector and one value vector per layer, per head.

- Per-token KV cache size $\approx 2 \times n_{\text{layers}} \times n_{\text{heads}} \times d_{\text{head}} \times \text{bytes/element}$, where the 2 counts K and V (queries are never stored)
- Multiply by sequence length for the full-sequence cache of one request
- Multiply again by batch size for all concurrent requests being served at once

Example: KV cache vs. weights

Toy model: 32 layers, 32 heads, head dim 128, fp16 (2 bytes), sequence length 2048.

- Per-token, per-request: $2 \times 32 \times 32 \times 128 \times 2 \text{ bytes} \approx 524\text{KB}$
- For one 2048-token sequence: $524\text{KB} \times 2048 \approx \mathbf{1.07GB}$
- For a batch of 64 concurrent sequences at that length: $\approx \mathbf{69GB}$
- Compare to this model's weights at, say, 13B params in fp16 (~26GB): the cache alone can exceed the weights

Example: cache growth with batch size

Batch size	Seq. length	Approx. KV cache
1	2048	~1.1GB
8	2048	~8.6GB
64	2048	~69GB
64	8192	~275GB

Two knobs — more users, or longer context — both push the cache up linearly. Serving systems must budget for both.

Prefill fills the cache, decode extends it

- **Prefill**: all prompt tokens are processed in parallel, writing their K and V in one pass; this sets the **time to first token** (TTFT): the wait before the first output appears
- **Decode**: each step appends one K, V pair per layer and reads the whole cache
- The cache lives until the request finishes, then its memory is freed
- Free cache memory caps how many requests fit at once, so it caps the batch size and the throughput

Managing cache memory: paged allocation

- Naive allocation reserves one contiguous block per request, sized for the **maximum** length, and most of it sits unused
- **PagedAttention** (vLLM) stores the cache in small fixed-size blocks, allocated on demand and tracked in a block table, like virtual-memory paging in an OS
- Waste drops to at most one partly filled block per request, so many more requests fit
- Requests sharing a prompt prefix (such as a system prompt) can share its blocks: **prefix caching**

Check your understanding

- Why can token i's key and value be reused at every later step, while its query cannot be used again?
- With a cache, per-step attention is still $O(n)$. Why can't that remaining cost be removed?
- Why does the KV cache grow **linearly** with both sequence length and batch size, while weight memory does neither?
- Which terms in the cache-size formula can a serving system change at run time, and what does shrinking each one cost?

Section recap

- Causal attention makes every past token's k and v permanent, so caching them cuts per-step attention from $O(n^2)$ to $O(n)$
- Queries are used once, by their own position, so they are never cached
- The cost moves from compute to memory: cache size scales with layers, heads, sequence length, and batch size
- Paged allocation and prefix sharing keep that memory in check
- Next: when weights and cache do not fit on one device, split the model across devices



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Parallelism strategies

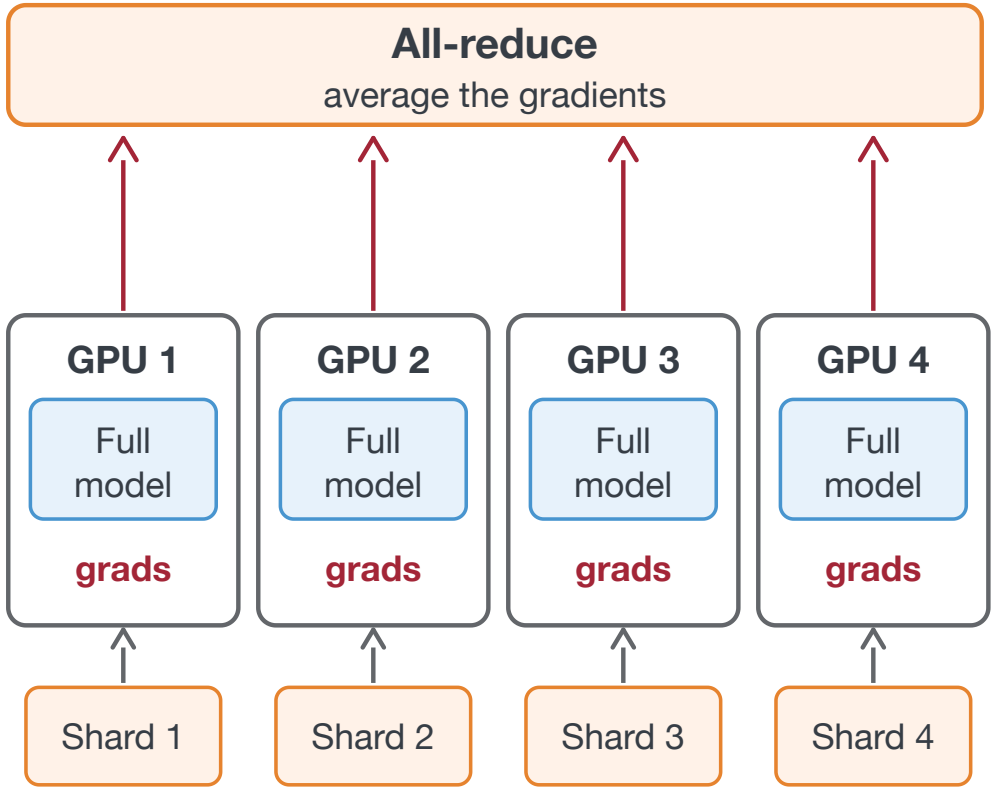
Data, tensor, pipeline

Why parallelism at all

- Two separate reasons to split work across multiple devices, and they call for different strategies
- **Reason 1 — speed**: even a model that fits on one device trains/serves faster split across many
- **Reason 2 — capacity**: some models (the 70B/175B examples above) simply do not fit in one device's memory, full stop
- Data parallelism addresses reason 1; tensor and pipeline parallelism address reason 2 (and help with reason 1 too)

Data parallelism

- Replicate the **entire** model on every device
- Each device processes a different batch shard and computes its own gradients
- Gradients are averaged across devices before each optimizer update
- Does not help when the model is too big for one device



Different data on each GPU, identical weights

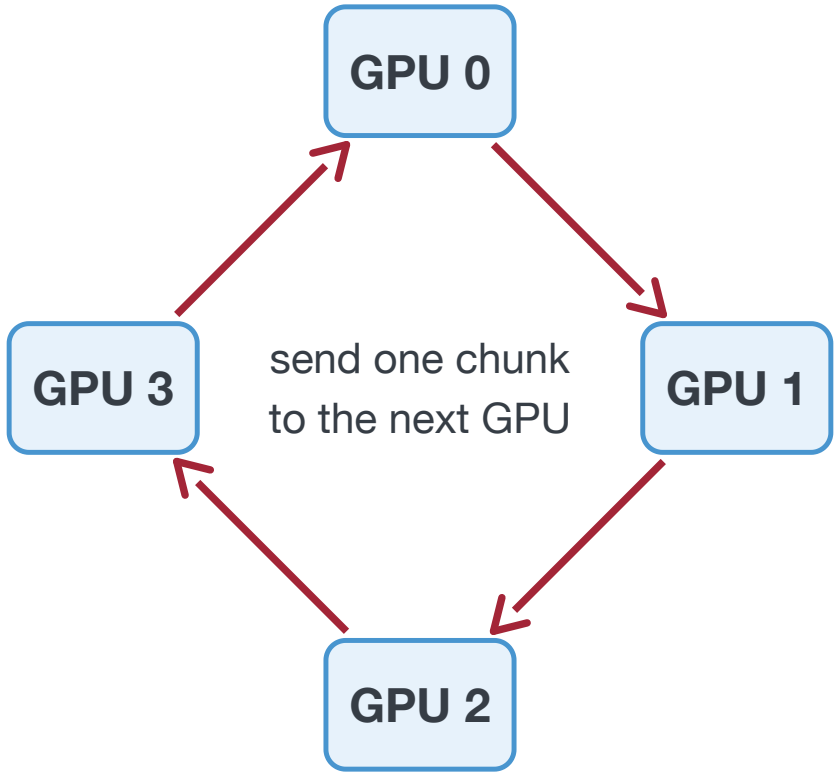
Example: data parallelism at batch scale

- Global batch size 256, split across 8 GPUs → each GPU processes a local batch of 32
- Each GPU computes its own forward pass, loss, and gradients on its 32 examples
- An **all-reduce** operation averages the 8 gradient tensors so every GPU ends the step with the identical, averaged gradient
- Every GPU then takes the **same** optimizer step — this is why replicas stay identical throughout training

Ring all-reduce, briefly

How the gradient averaging is implemented efficiently.

- Naive all-reduce sends everything to one coordinator, which becomes a bottleneck
- **Ring all-reduce**: devices form a logical ring; each passes a chunk to its neighbor, accumulating along the way
- Per-device communication depends on the data size, not the number of devices



The limits of data parallelism

- Recall lecture 2's optimizers (SGD, Adam): data parallelism scales the **data** side of gradient descent, not the **model** side
- Every device still needs the full weights, full gradients, and full optimizer state resident in memory
- For a model whose weights alone need 140GB, data parallelism alone cannot help — you need to split the **model**, not just the batch
- This motivates tensor and pipeline parallelism, covered next

Tensor parallelism

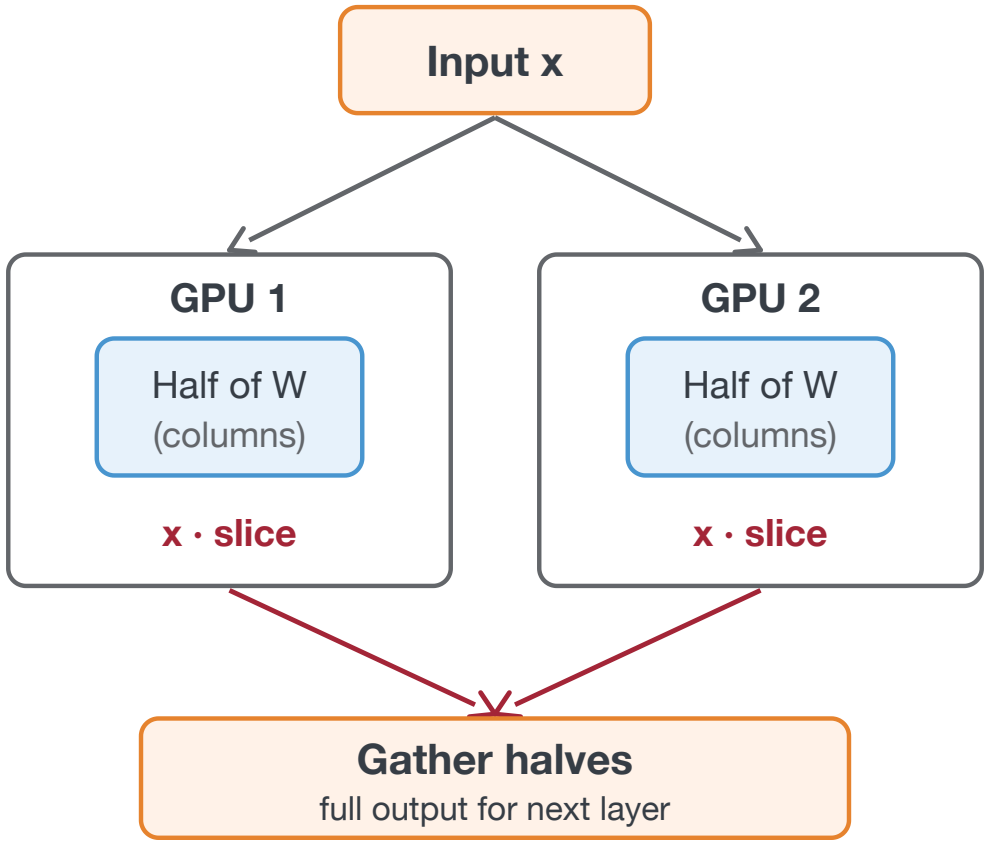
Split individual matrix multiplications across devices.

- A weight matrix is partitioned (e.g. by columns) across GPUs
- Each GPU computes its slice of the matrix multiply; results are combined with a communication step
- Reduces per-device memory **and** lets one layer's compute span multiple devices
- Cost: frequent communication between devices, so it works best **within** a fast-interconnect machine

Example: splitting one matmul

Weight matrix W of shape $(d_{\text{model}}, d_{\text{ff}})$, split by columns across 2 GPUs.

- Each GPU holds half the columns of W
- Both GPUs get the **same** input x
- Each computes xW_{slice}: half of the output columns
- An all-gather (or all-reduce) rebuilds the full output for the next layer



Same input, different slice of the weights

Example: splitting attention heads

- Attention has a natural place to split: heads are already independent computations
- With H heads split across 2 GPUs, each GPU computes attention for $H/2$ heads entirely locally
- Only the final output projection needs a communication step to combine head outputs
- This is why tensor-parallel attention is comparatively cheap to communicate, relative to splitting the feed-forward matrices

Why tensor parallelism needs a fast interconnect

- Every layer requires a communication step — this happens **dozens of times per forward pass**, not once
- Communication latency, not just bandwidth, adds up across every layer
- Fast intra-node links (e.g. NVLink) make this affordable; going **across** machines over a slower network usually is not
- Rule of thumb: keep tensor parallelism inside one machine; use other strategies to span machines

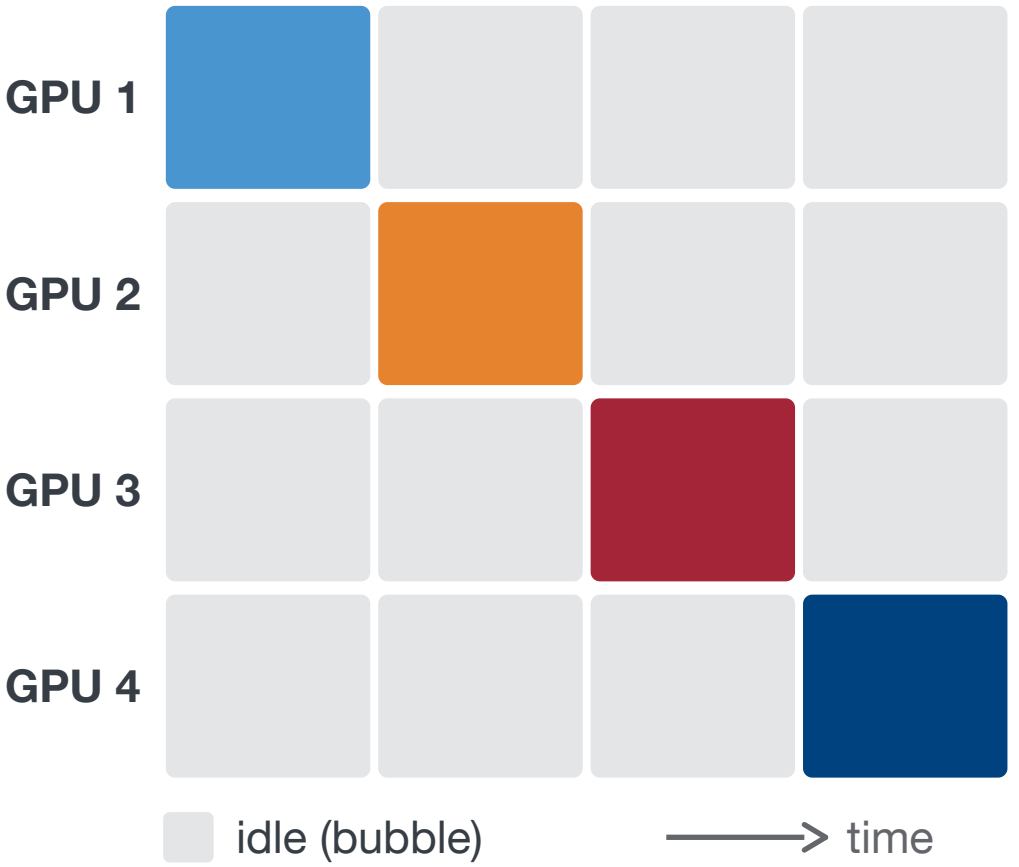
Pipeline parallelism

Split the model's **layers** across devices instead of splitting each layer.

- Device 1 holds layers 1-8, device 2 holds layers 9-16, and so on
- Data flows through the pipeline stage by stage, like an assembly line
- Naive version leaves early stages idle while later stages finish — **pipeline bubbles**
- Micro-batching: split each batch into smaller chunks that flow through the pipeline back-to-back, filling the bubbles

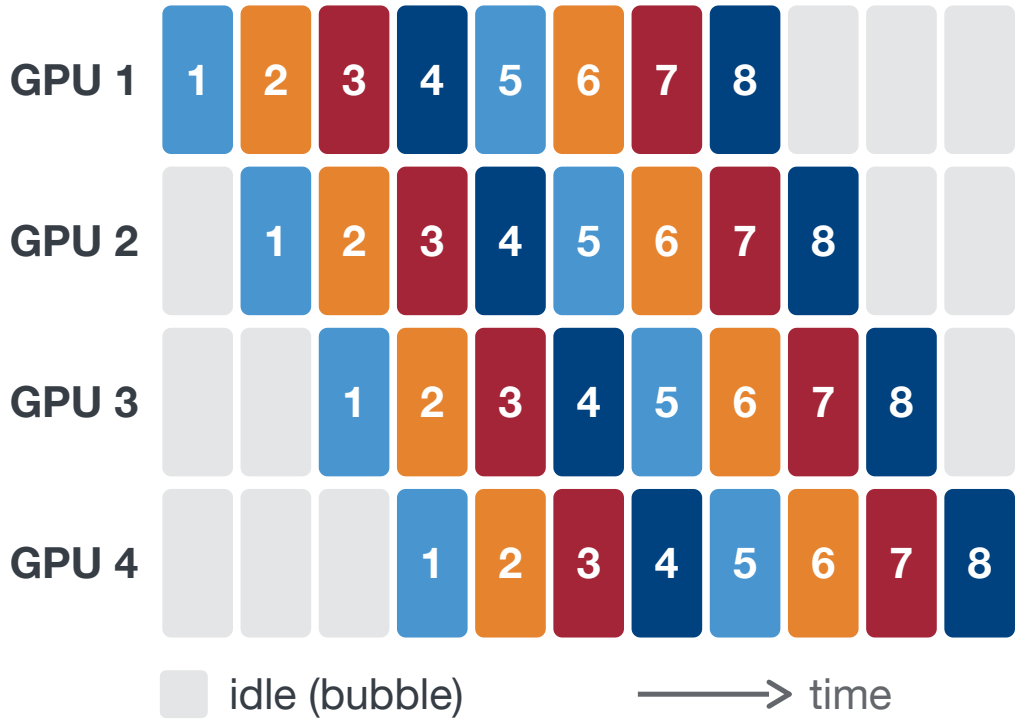
Example: a 4-stage pipeline without micro-batching

- 4 devices, one layer-group each; one batch flows through
- Device 1 works while 2, 3, 4 idle; then 2 works while the rest idle; and so on
- The idle “bubble” is roughly 3/4 of the wall-clock time
- Nothing overlaps: a poor use of 4 devices



Example: filling the bubble with micro-batches

- Split the batch into 8 micro-batches and feed them back-to-back
- While device 1 takes micro-batch 2, device 2 already works on micro-batch 1
- Bubble fraction: $\frac{p-1}{m+p-1}$ for p stages, m micro-batches
- $p = 4, m = 8$: about $3/11 \approx 27\%$, down from ~75%

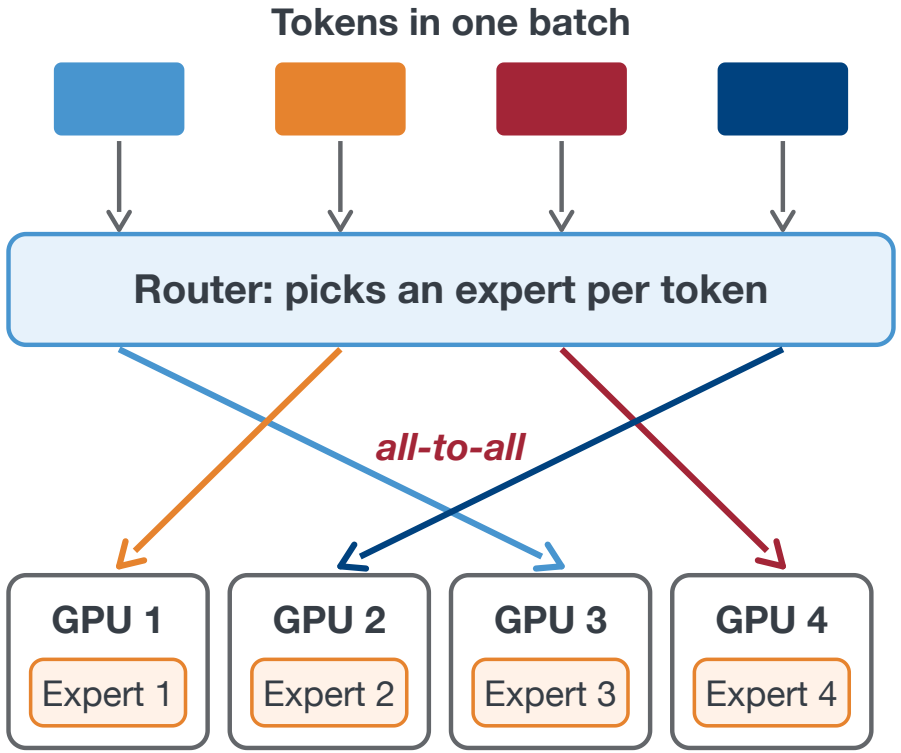


Sequence parallelism

- A fourth way to split: along the **sequence** dimension, for contexts too long for one device to hold the activations
- Different tokens of the same sequence are processed on different devices, with communication at attention layers where tokens must interact
- Complements tensor and pipeline parallelism rather than replacing them: used for very long context windows
- We won’t go deep here; the point is that “split the model” has more axes to cut along than you might expect

Expert parallelism

- For **mixture-of-experts (MoE)** models: each layer holds many feed-forward **experts**, and a router sends every token to only a few of them
- Put different experts on different devices; the router decides where each token must travel
- Two **all-to-all** exchanges per MoE layer: dispatch tokens to their experts, then bring the results back
- Gain: more parameters, same compute per token
- Cost: all-to-all traffic, and load imbalance if tokens favor one expert



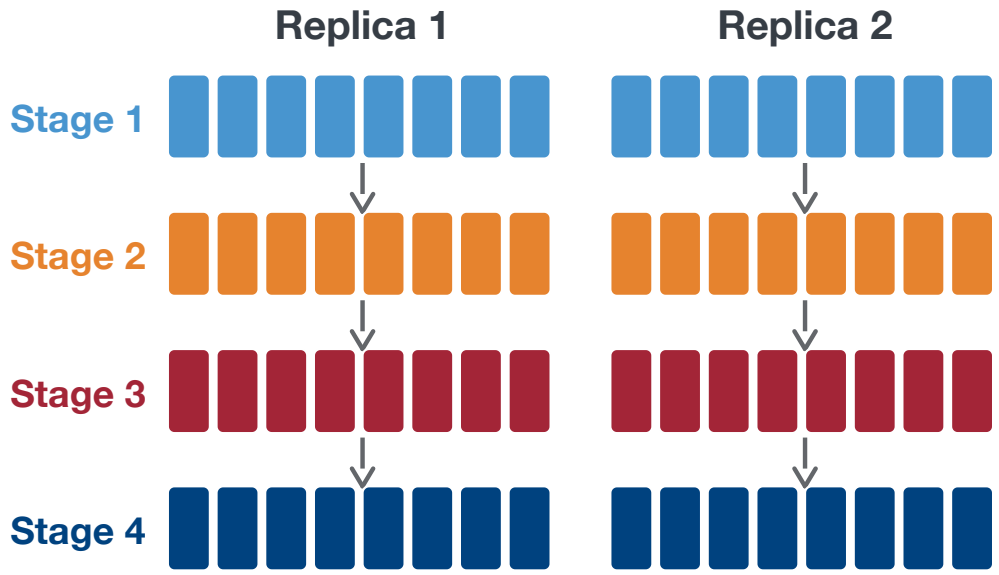
Each GPU holds different experts; tokens travel to them

Combining strategies: 5D parallelism

- Real large-model training and serving combines all five axes: data, tensor, pipeline, sequence, and expert parallelism
- Tensor parallelism within a machine (fast interconnect), pipeline across machines, data parallelism across whole replicas
- Sequence parallelism usually shares the tensor-parallel group; expert parallelism spreads experts across devices
- The combination is sometimes called **5D parallelism**; any axis can have size 1
- Choosing the split is itself an engineering problem, shaped by network topology and model architecture

Example: mapping a cluster

- 64 GPUs, a dense model: three of the five axes (sequence and expert have size 1)
- 8-way tensor parallelism: one model copy spans the 8 GPUs of a fast-interconnect node
- 4-way pipeline parallelism: 4 such groups form 4 stages
- Remaining $64/(8 \times 4) = 2$-way data parallelism: two replicas



One row = 8 GPUs in one node: 8-way tensor parallel

Rows top to bottom = 4-way pipeline

Two replicas = 2-way data parallel

8 × 4 × 2 = 64 GPUs

Choosing a parallelism strategy

Situation	Reach for
Model fits on one device, just want speed	Data parallelism
One layer's weights alone don't fit	Tensor parallelism (inside a fast-interconnect node)
Whole model doesn't fit, but each layer does	Pipeline parallelism (with micro-batching)
Very long context, activations don't fit	Sequence parallelism
Mixture-of-experts model, experts don't fit	Expert parallelism
Frontier-scale model on a large cluster	All of the above, combined (5D parallelism)

Check your understanding

- Why does data parallelism fail to solve the “model doesn’t fit in memory” problem, even with unlimited devices?
- In the micro-batching example, what happens to the bubble fraction as the number of micro-batches m grows very large relative to pipeline stages p?
- Why is tensor parallelism usually confined to a single machine, while pipeline parallelism can span machines?

Section recap

- Data parallelism scales speed by replicating; tensor and pipeline parallelism scale capacity by splitting the model itself
- Tensor parallelism splits **within** a layer (needs fast interconnect); pipeline parallelism splits **across** layers (tolerates slower links, but needs micro-batching to avoid bubbles)
- Production systems compose all of these into one 5D grid of devices
- Next: even with parallelism, training and serving still need memory optimization tricks



STEVENS
INSTITUTE OF TECHNOLOGY
1870

Memory optimization

Checkpointing · Precision · Quantization

The memory budget during training

Training a model needs more than just the weights.

- **Weights**: the parameters themselves
- **Gradients**: one value per parameter, computed each backward pass
- **Optimizer state**: e.g. Adam's momentum and variance terms — recall lecture 2's optimizers, now paid for in memory, not just compute
- **Activations**: intermediate values kept around for the backward pass

Example: Adam's memory overhead

For a 7B-parameter model, fp32 training, using Adam (from lecture 2's optimizer survey).

Component	Size
Weights	~28GB
Gradients	~28GB
Adam momentum term	~28GB
Adam variance term	~28GB
Total (excluding activations)	**~112GB**

Optimizer state alone can be 2-4× the size of the weights for Adam — this is often the **biggest** line item, not the weights.

Data parallelism's hidden cost

- Recall: data parallelism replicates the **entire** model, including all of the memory above, on every device
- 8-way data parallelism on the 7B example above means $8 \times 112\text{GB} = 896\text{GB}$ of redundant state across the cluster, storing the **same** information 8 times
- Adding replicas buys throughput, not headroom: each GPU still has to fit the full training state
- That is why the per-GPU tricks that follow matter even on a large cluster

Gradient accumulation: a simple trick

- Solves a specific problem: wanting a larger **effective** batch size than memory allows
- Run several forward/backward passes on small micro-batches, **summing** gradients instead of applying them immediately
- Only after accumulating over several micro-batches, take one optimizer step
- Costs no extra memory for optimizer state: it trades wall-clock time for a larger effective batch

Activation checkpointing

- Storing every intermediate activation for backpropagation is memory-hungry, especially for very deep networks
- **Activation checkpointing**: discard most activations during the forward pass, recompute them during the backward pass
- Trades extra compute (a second forward pass, restricted to the parts you discarded) for a large memory saving
- Useful when memory, not compute, is the bottleneck — very common at large model scale

Example: checkpointing tradeoff

- Without checkpointing: memory for activations grows linearly with layer count L
- With checkpointing every layer: memory grows like $\sqrt{L}$ instead, at the cost of roughly one extra forward pass ($\approx 33\%$ more compute for that pass)
- A common sweet spot: checkpoint at every $\sqrt{L}$-th layer, balancing memory savings against the extra recomputation

Mixed precision training

- Represent weights and activations in 16-bit (or lower) floating point instead of 32-bit
- Halves memory for those tensors and speeds up matrix multiplies on modern GPUs, which have dedicated low-precision hardware paths
- Keep a 32-bit **master copy** of weights for the optimizer update, so small gradient updates don’t get lost to rounding when applied to 16-bit weights

fp16 vs. bf16

	fp16	bf16
Exponent bits	**5**	8 (same as fp32)
Mantissa bits	**10**	7
Dynamic range	**Narrow — can overflow/underflow**	Wide — matches fp32's range
Needs loss scaling?	**Usually yes**	Usually no

bf16 sacrifices some precision (mantissa) to keep fp32's **range** (exponent) — for training stability, range matters more than precision in practice.

Why loss scaling exists

- fp16's narrow exponent range means small gradient values can underflow to exactly zero before they're even used
- **Loss scaling**: multiply the loss by a large constant before the backward pass, so gradients land in a representable range; divide back out before the optimizer step
- bf16's wider exponent range means this trick is usually unnecessary — one reason bf16 has become the more common default for large-model training

Quantization

- Reduce numeric precision further, most often for **inference** rather than training: 8-bit, 4-bit, or lower
- Shrinks memory footprint and can speed up serving, at some cost to accuracy
- **Post-training quantization**: quantize an already-trained model's weights, no retraining needed, fastest to apply
- **Quantization-aware training**: simulate low-precision effects **during** training, so the model adapts to it — costs more upfront, preserves more accuracy

Example: quantization memory savings

For a 70B model, weights only.

Precision	Bytes/param	Total weight memory
fp16	2	~140GB
int8	1	~70GB
int4	0.5	~35GB

A 70B model that needed 2+ high-end GPUs at fp16 can sometimes fit on a **single** GPU at int4 — at some, task-dependent, accuracy cost.

Putting it together: a serving memory budget

For a 70B model, batch of 32, sequence length 4096, quantized to int8.

- Weights (int8): ~70GB
- KV cache (fp16, batch 32 at 4096 tokens): still large, and it scales with batch size and context length, so it needs its own line in the budget
- The serving engineer's job: fit weights + KV cache + activation buffers into the available GPU memory, choosing precision and parallelism to make the numbers work
- Every technique in this section is a dial on the same budget — none of them is free, all of them trade something for memory

Check your understanding

- Why is optimizer state, not weights, often the largest single memory consumer during training with Adam?
- What's the practical difference between mixed-precision training and quantization for serving? Could you use both on the same model?
- Why does bf16 usually avoid needing loss scaling, when fp16 does not?

Section recap

- Training's memory budget has four components (weights, gradients, optimizer state, activations), and each has its own set of tricks
- Checkpointing trades compute for activation memory; mixed precision and quantization trade numeric precision for memory
- None of these tricks is free: each is a different point on a memory/compute/accuracy tradeoff
- Next week: from engineering a trained model to **learning from reward** with reinforcement learning

Looking ahead: teams and reinforcement learning

- **Project teams form around now** — start thinking early about a problem you'd actually like to solve, research or application
- Everything covered today — parallelism, memory tricks, RAG, agents — is **engineering** on top of an already-trained model
- Lecture 7's RLHF was reinforcement learning **applied to** language models, one narrow instance of a much bigger idea
- Starting next week, we turn to **reinforcement learning** itself: how an agent learns from reward, from first principles

Two lectures from now, the RLHF you saw last week — and the plan-act-observe loop you saw today — will both look like special cases of one general framework.



STEVENS
INSTITUTE OF TECHNOLOGY
1870

THANK **YOU**

Stevens Institute of Technology
1 Castle Point Terrace, Hoboken, NJ 07030